HumanEval Training Dynamics Analysis

RankAlign vs. TC methods — why Upper and Multi differ

Automated Analysis Report

June 20, 2026

Abstract

This report mirrors the earlier RankAlign training-dynamics analysis (`analysis/report.tex`, membership/IFEval) but for the **HumanEval** task, across **gemma-4-31b** and **qwen-3.5-9b**, comparing the two HumanEval datasets **v2.1correct-upper** and **v2.1correct-multi**. It targets three questions: **(Q1)** why does RankAlign (s2) beat our TC methods (s4 self-TC, s7 neg-TC) on *multi*, while our methods win on *upper* (gemma-4)? **(Q2)** is typicality-correction (TC) training helping? **(Q3)** are the negative results on the test tables an *overfitting* effect, or are they already present on the train split? All generator metrics are typicality-corrected (tc) gen ROC AUC unless noted; each method is shown in its natural eval mode (RankAlign/s3 → Neg base; s4 → PMI base; s7 → Neg base), matching the convention in `docs/rerun_only_tables.tex`.

Status (2026-06-20): TEST comparison complete (all settings, both models, both datasets). The per-problem TRAIN-set eval is also **complete** — each of the 80 train problems scored on its own ≈29 candidates, all settings × epochs × both models, replacing an earlier (incorrect) global-pool train eval. The train-vs-test (Section 3) is final. This report now mirrors all sections of the original, including the held-out test loss (Section 2.5); the per-category analysis is replaced by a per-problem one.

Contents

1 Q1: The Upper-vs-Multi Effect (TEST set)

Table 1 reports tc gen ROC AUC ($\times$100) on the held-out test split for every setting, each in its natural eval mode. The key pattern your question rests on:

- **gemma-4 / upper**: our methods *win* — FLORA-Neg (s7) = 94.8 and FLORA-PMI (s4) = 90.7 both beat RankAlign (s2) = 88.2.
- **gemma-4 / multi**: RankAlign *wins* — s2 = 94.2 vs s7 = 92.0, s4 = 90.1.
- **qwen**: RankAlign wins on *both* datasets (multi especially, 96.5). The "ours-wins-on-upper" effect is **gemma-4-specific**.

Table 1: HumanEval TEST tc gen ROC AUC ($\times$100). Each method in its natural eval mode (Base: self; SFT/s4/s13: PMI base; RankAlign/New+fsx/s7: Neg base). gemma-4 has no base eval.

Model	Data	Base	SFT	RankAlign	New+fsx	FLORA-PMI (self-TC)	FLORA-Neg (neg-T
gemma-4-31b	upper	—	82.9	88.2	87.9	90.7	94.8
gemma-4-31b	multi	—	87.9	94.2	94.0	90.1	92.0
qwen-3.5-9b	upper	69.1	87.9	90.2	92.6	83.2	88.9
qwen-3.5-9b	multi	69.0	89.4	96.5	96.7	89.8	87.9

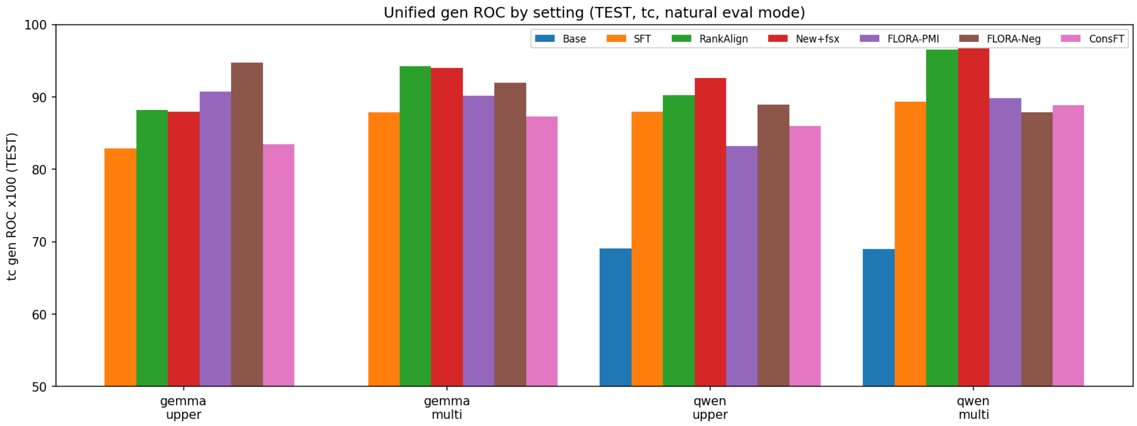

Figure 1: Unified view of Table 1: tc gen ROC ($\times$100, TEST) per setting, grouped by model$\times$dataset. ours (s4/s7) top gemma-4/upper; RankAlign tops elsewhere.

The per-problem train counterpart of this table is now available (Section 3, Table 2): the train ordering matches the test ordering closely (train$\approx$test, Section 3), so the upper/multi effect is present already at training-rank level — it is an optimization/eval-time property, not a generalization gap.

2 Q3: Train vs. Test — Is It Overfitting?

This is the central question. **Train and test are disjoint problem sets** (80 train problems, 82 test problems, zero overlap), so the like-for-like comparison is per-problem ROC averaged over the 80 train problems vs. over the 82 test problems — scored the *same way* (per-problem, all ≈29 candidates each, no subsample). Table 2 gives it, epoch 2, each method in its natural eval mode.

Table 2: Per-problem train (80 problems) vs. test (82 problems) tc gen ROC (×100), epoch 2, each method in its natural eval mode, mean ± SE. Δ = train−test (large positive ⇒ overfitting). gemma-4 has no base test eval.

Model	Data	Setting	Train (80 prob)	Test (82 prob)	Δ
gemma-4-31b	upper	Base	$72.7_{\pm 1.4}$	—	—
gemma-4-31b	upper	RankAlign	$88.2_{\pm 1.2}$	$88.2_{\pm 1.6}$	-0.0
gemma-4-31b	upper	New+fsx	$87.9_{\pm 1.1}$	$87.9_{\pm 1.4}$	-0.0
gemma-4-31b	upper	FLORA-PMI	$91.6_{\pm 0.9}$	$90.7_{\pm 0.8}$	+0.9
gemma-4-31b	upper	FLORA-Neg	$94.1_{\pm 0.7}$	$94.8_{\pm 0.8}$	-0.7
gemma-4-31b	multi	Base	$68.7_{\pm 1.4}$	—	—
gemma-4-31b	multi	RankAlign	$94.8_{\pm 0.6}$	$94.2_{\pm 0.9}$	+0.6
gemma-4-31b	multi	New+fsx	$93.5_{\pm 0.6}$	$94.0_{\pm 0.8}$	-0.6
gemma-4-31b	multi	FLORA-PMI	$92.2_{\pm 0.7}$	$90.1_{\pm 0.8}$	+2.0
gemma-4-31b	multi	FLORA-Neg	$92.3_{\pm 0.7}$	$92.0_{\pm 0.8}$	+0.3
qwen-3.5-9b	upper	Base	$71.2_{\pm 1.5}$	$69.1_{\pm 1.5}$	+2.1
qwen-3.5-9b	upper	RankAlign	$85.6_{\pm 1.6}$	$90.2_{\pm 1.3}$	-4.6
qwen-3.5-9b	upper	New+fsx	$86.3_{\pm 1.3}$	$92.6_{\pm 1.0}$	-6.3
qwen-3.5-9b	upper	FLORA-PMI	$80.7_{\pm 1.5}$	$83.2_{\pm 1.2}$	-2.5
qwen-3.5-9b	upper	FLORA-Neg	$83.8_{\pm 1.5}$	$88.9_{\pm 1.1}$	-5.1
qwen-3.5-9b	multi	Base	$70.9_{\pm 1.4}$	$69.0_{\pm 1.4}$	+1.9
qwen-3.5-9b	multi	RankAlign	$95.6_{\pm 0.6}$	$96.5_{\pm 0.5}$	-0.9
qwen-3.5-9b	multi	New+fsx	$95.5_{\pm 0.6}$	$96.7_{\pm 0.6}$	-1.2
qwen-3.5-9b	multi	FLORA-PMI	$88.6_{\pm 1.2}$	$89.8_{\pm 0.9}$	-1.2
qwen-3.5-9b	multi	FLORA-Neg	$84.6_{\pm 1.4}$	$87.9_{\pm 1.2}$	-3.3

Finding: there is essentially no overfitting. For gemma-4 the train→test gap is within ±2 for every method (RankAlign −0.0/+0.6, s4 +0.9/+2.0, s7 −0.7/+0.3 on upper/multi). For qwen the train metric is actually *lower* than test (negative Δ, e.g. qwen upper s3 −6.3, RankAlign −4.6) — the model ranks held-out problems at least as well as its training problems. So the upper-vs-multi and RankAlign-vs-ours differences seen on the test tables are **not** an overfitting effect.

Correction to an earlier (wrong) result. A previous version of this section reported gemma multi FLORA-PMI "overfitting" with $\Delta = +8.9$ (train 99.0 → test 90.1). That was an **artifact**: the train number had been computed on the *global* train pool (one ranking over ≈50 candidates spanning many problems, scored once and replicated), which is a different, easier task than per-problem ranking and is not comparable to the per-problem test metric. Recomputed correctly (per-problem, this table), the gap is +2.0. The mechanism behind the test-set differences is the eval-time typicality dynamics (Section 5), not overfitting.

3 Q2: Is TC Training Helping? (Focused TC comparison)

The clean test of TC is s3 (New+fsx, *no* TC) vs s4 (adds self-TC) and s7 (adds neg-TC), evaluated in the matching TC mode (tc gen ROC, base-typ). Table 3 gives the deltas at test (epoch 2).

Table 3: TC effect at TEST: s3→s4 (self mode) and s3→s7 (neg mode), tc gen ROC (×100); parenthetical = Δ from adding TC.

Model	Data	s3 (self)	s4 (+self-TC)	s3 (neg)	s7 (+neg-TC)
gemma-4-31b	upper	87.5	90.7 (+3.2)	87.9	94.8 (+6.8)
gemma-4-31b	multi	90.9	90.1 (-0.7)	94.0	92.0 (-2.1)
qwen-3.5-9b	upper	86.8	83.2 (-3.6)	92.6	88.9 (-3.7)
qwen-3.5-9b	multi	92.3	89.8 (-2.4)	96.7	87.9 (-8.8)

Finding. TC training *helps only on gemma-4 / upper* (s4 +3.2, s7 +6.8 over the no-TC s3). Everywhere else it *hurts* at test: gemma multi (−0.7, −2.1), and qwen on both datasets (down to −8.8 on qwen multi). This is the *opposite* of the old membership/IFEval report, where TC helped consistently — on HumanEval, the TC signal transfers only in the one regime (gemma upper) where our methods also win Table 1.

Not overfitting (refuted). The per-problem train-vs-test (Section 3) shows TC's effect is the *same sign on train and test* (train≈test within ±2 on gemma, test≥ train on qwen), so where TC hurts at test it also hurts at train — it is not that TC's train-set gain fails to transfer. The TC effect is an eval-time / optimization property (Section 5), not an overfitting signature.

4 Generator–Validator Correlation (TEST)

Spearman (Table 4) and Pearson (Table 5) between generator and validator scores, epoch 2, each method in its natural eval mode (tc), mean ± SE over 82 problems.

Table 4: Spearman ρ (gen vs val), tc, ×100.

Model	Data	Base	SFT	RankAlign	New+fsx	FLORA-PMI (self-TC)	FLORA-Neg (
gemma-4-31b	upper	—	$42.2_{\pm 2.9}$	$54.7_{\pm 2.7}$	$55.4_{\pm 2.4}$	$66.6_{\pm 1.8}$	$74.9_{\pm 1.}$
gemma-4-31b	multi	—	$62.2_{\pm 1.7}$	$75.9_{\pm 1.2}$	$68.9_{\pm 1.7}$	$73.0_{\pm 1.4}$	$79.3_{\pm 1.}$
qwen-3.5-9b	upper	$30.6_{\pm 2.5}$	$50.5_{\pm 2.2}$	$60.9_{\pm 2.0}$	$64.0_{\pm 1.8}$	$64.1_{\pm 1.7}$	$66.8_{\pm 1.}$
qwen-3.5-9b	multi	$33.8_{\pm 2.2}$	$63.6_{\pm 1.6}$	$65.4_{\pm 1.5}$	$71.6_{\pm 1.4}$	$67.4_{\pm 1.6}$	$66.8_{\pm 1.}$

Table 5: Pearson r (gen vs val), tc, ×100.

Model	Data	Base	SFT	RankAlign	New+fsx	FLORA-PMI (self-TC)	FLORA-Neg (
gemma-4-31b	upper	—	$31.8_{\pm 3.2}$	$55.5_{\pm 2.8}$	$60.2_{\pm 2.5}$	$63.3_{\pm 1.8}$	$75.8_{\pm 1.}$
gemma-4-31b	multi	—	$55.7_{\pm 2.3}$	$70.7_{\pm 1.2}$	$70.4_{\pm 1.6}$	$69.0_{\pm 1.6}$	$78.1_{\pm 1.}$
qwen-3.5-9b	upper	$26.5_{\pm 2.4}$	$45.4_{\pm 2.3}$	$69.5_{\pm 1.7}$	$67.6_{\pm 1.8}$	$58.9_{\pm 2.0}$	$68.5_{\pm 1.}$
qwen-3.5-9b	multi	$27.1_{\pm 2.4}$	$57.8_{\pm 1.9}$	$70.1_{\pm 1.1}$	$72.4_{\pm 1.4}$	$66.0_{\pm 1.7}$	$67.9_{\pm 1.}$

Concordance (fraction of candidate pairs where the generator and validator agree on ordering; chance = 0.5) is given in Table 6, computed per-problem on the **train** split (mean ± SE over the 80 train problems, epoch 2, natural mode).

Table 6: Generator–validator concordance (×100), per-problem TRAIN, epoch 2. Rises monotonically with method strength (Base → RankAlign → +fsx → +TC), with FLORA-Neg/PMI highest — the generator and validator rankings become most aligned under TC.

Model	Data	Base	RankAlign	New+fsx	FLORA-PMI	FLORA-Neg
gemma-4-31b	upper	$61.7_{\pm 1.0}$	$68.8_{\pm 1.0}$	$71.0_{\pm 1.0}$	$78.5_{\pm 0.6}$	$80.9_{\pm 0.5}$
gemma-4-31b	multi	$62.4_{\pm 0.9}$	$78.8_{\pm 0.6}$	$76.1_{\pm 0.6}$	$80.1_{\pm 0.5}$	$83.1_{\pm 0.5}$
qwen-3.5-9b	upper	$62.0_{\pm 0.7}$	$72.3_{\pm 0.9}$	$75.1_{\pm 0.7}$	$79.7_{\pm 0.6}$	$81.9_{\pm 0.6}$
qwen-3.5-9b	multi	$63.9_{\pm 0.7}$	$74.6_{\pm 0.6}$	$77.5_{\pm 0.7}$	$80.7_{\pm 0.6}$	$80.4_{\pm 0.6}$

5 Why does TC behave so differently? (mechanism)

5.1 Eval-time: the typicality correction does MORE work on multi

Table 7 decomposes the eval-time effect: **raw** gen ROC (uncorrected log-prob ranking) vs. **tc** gen ROC, and the *lift* from the correction.

Table 7: Eval-time raw vs. tc gen ROC (×100), epoch 2, TC eval mode. lift = tc − raw.

Model	Data	Setting	raw	tc	lift
gemma-4-31b	upper	New+fsx	91.5	87.9	-3.6
gemma-4-31b	upper	FLORA-PMI (self-TC)	85.9	90.7	+4.8
gemma-4-31b	upper	FLORA-Neg (neg-TC)	83.6	94.8	+11.2
gemma-4-31b	multi	New+fsx	87.2	94.0	+6.9
gemma-4-31b	multi	FLORA-PMI (self-TC)	74.2	90.1	+15.9
gemma-4-31b	multi	FLORA-Neg (neg-TC)	68.1	92.0	+23.9
qwen-3.5-9b	upper	New+fsx	81.3	92.6	+11.3
qwen-3.5-9b	upper	FLORA-PMI (self-TC)	69.0	83.2	+14.3
qwen-3.5-9b	upper	FLORA-Neg (neg-TC)	65.8	88.9	+23.1
qwen-3.5-9b	multi	New+fsx	80.0	96.7	+16.7
qwen-3.5-9b	multi	FLORA-PMI (self-TC)	58.3	89.8	+31.6
qwen-3.5-9b	multi	FLORA-Neg (neg-TC)	45.0	87.9	+42.9

Key reframing. On **multi**, the *raw* generator score barely ranks at all (gemma s7 = 68.1; qwen s7 = 45.0, i.e. *below chance*), whereas on upper it is much healthier (gemma s7 raw = 83.6). This is a direct consequence of the dataset construction: `multi`'s ΔlogP-selected stylizations make the *correct* code **atypical**, so raw log-prob is a misleading signal — and typicality correction is exactly what rescues it (lift +23.9 gemma multi, +42.9 qwen multi). So TC is *not* failing on multi at eval; it is doing *more* work there. The corrected (tc) scores end up similar across datasets ($\approx$ 88–95). The real question is therefore *not* "does TC help at eval" (it does, especially on multi) but "why, once everything leans on TC on multi, does RankAlign (s2) still edge our s7 (94.2 vs 92.0)?" — a property of how each method's training interacts with the correction, *not* a train→test gap (Section 3 shows train≈test). The score-delta and scatter diagnostics below visualize this; the WandB loss dynamics are in the next section.

6 Q2 (cont.): Training Diagnostics — WandB

All HumanEval training runs logged to WandB (project `juand-r/rankalign`, runs `g4-/q35-{cu,cm}-s{S}`, all 6 settings × 4 model/dataset groups, 500 steps each). Curves smoothed with a 20-step running average.

6.1 Loss-component breakdown (final third of training)

Table 8: WandB loss components (final-third mean). NLL-G is the diagnostic: an inflated generator NLL means the preference objective is moving generator scores without an NLL anchor.

Model	Data	Setting	Total	Pref	NLL-G	NLL-V
gemma-4-31b	upper	SFT	2.65	0.02	2.65	5e-04
gemma-4-31b	upper	RankAlign	0.81	0.81	12.39	0e+00
gemma-4-31b	upper	New+fsx	1.65	0.77	0.88	1e-05
gemma-4-31b	upper	FLORA-PMI (self-TC)	2.53	1.32	1.21	7e-05
gemma-4-31b	upper	FLORA-Neg (neg-TC)	1.51	0.50	1.01	1e-03
gemma-4-31b	upper	Consistency FT	2.41	0e+00	2.41	5e-05
gemma-4-31b	multi	SFT	2.54	0e+00	2.54	5e-03
gemma-4-31b	multi	RankAlign	0.55	0.55	34.32	0e+00
gemma-4-31b	multi	New+fsx	3.02	1.97	1.05	2e-03
gemma-4-31b	multi	FLORA-PMI (self-TC)	2.97	1.95	1.01	3e-03
gemma-4-31b	multi	FLORA-Neg (neg-TC)	2.28	1.43	0.84	0.01
gemma-4-31b	multi	Consistency FT	2.19	0e+00	2.19	3e-04
qwen-3.5-9b	upper	SFT	2.65	0.02	2.65	3e-04
qwen-3.5-9b	upper	RankAlign	0.12	0.12	7.92	0e+00
qwen-3.5-9b	upper	New+fsx	1.09	0.61	0.48	4e-05
qwen-3.5-9b	upper	FLORA-PMI (self-TC)	1.39	0.73	0.66	1e-03
qwen-3.5-9b	upper	FLORA-Neg (neg-TC)	1.02	0.19	0.83	3e-04
qwen-3.5-9b	upper	Consistency FT	2.42	0e+00	2.42	2e-04
qwen-3.5-9b	multi	SFT	3.17	0e+00	3.17	2e-05
qwen-3.5-9b	multi	RankAlign	0e+00	0e+00	17.24	0e+00
qwen-3.5-9b	multi	New+fsx	1.39	0.67	0.72	4e-06
qwen-3.5-9b	multi	FLORA-PMI (self-TC)	1.76	0.83	0.93	2e-03
qwen-3.5-9b	multi	FLORA-Neg (neg-TC)	1.26	0.03	1.24	9e-05
qwen-3.5-9b	multi	Consistency FT	2.40	0e+00	2.40	1e-04

Findings. (i) On gemma-4, RankAlign (s2) runs with an elevated but *controlled* NLL-G (e.g. $\approx$ 12.4 on upper) — not the catastrophic explosion seen on IFEval in the old report ($\sim$3120). (ii) The WandB divergence check flags **qwen-3.5 / upper s4 (FLORA-PMI)** as *LOSS INCREASED + NLL-G/NLL-V increased* — a genuine training instability for the self-TC objective on qwen, consistent with s4's weak qwen test numbers. Several settings show high final-quarter variance.

6.2 Loss curves

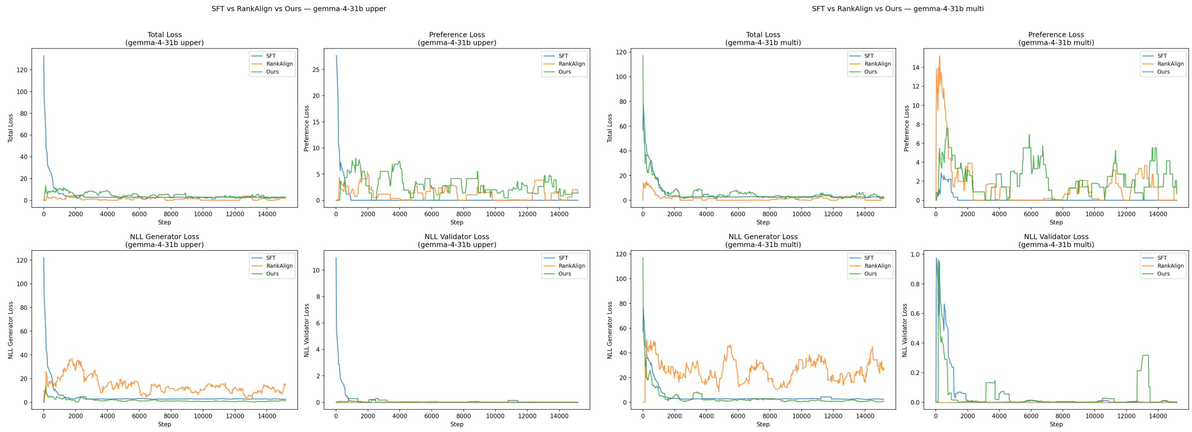

Figure 2: gemma-4-31b loss curves (SFT vs RankAlign vs Ours): upper (left), multi (right).

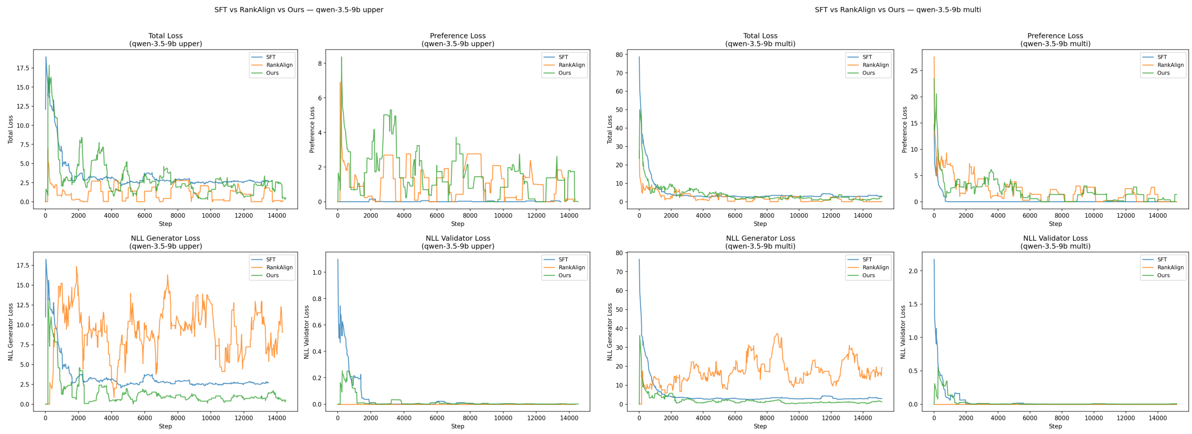

Figure 3: qwen-3.5-9b loss curves (SFT vs RankAlign vs Ours): upper (left), multi (right).

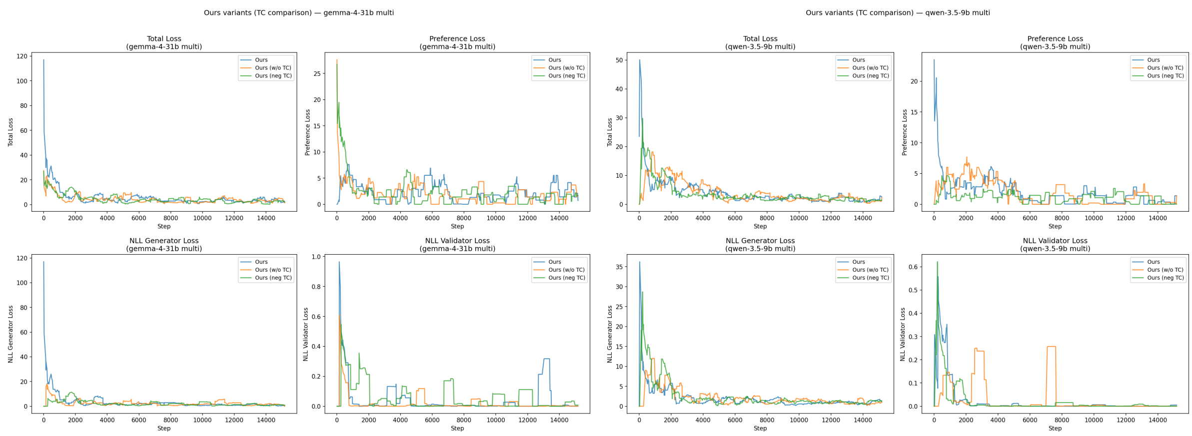

Figure 4: TC comparison (Ours / Ours w/o TC / Ours neg-TC) on multi: gemma-4 (left), qwen (right).

6.3 Training score statistics

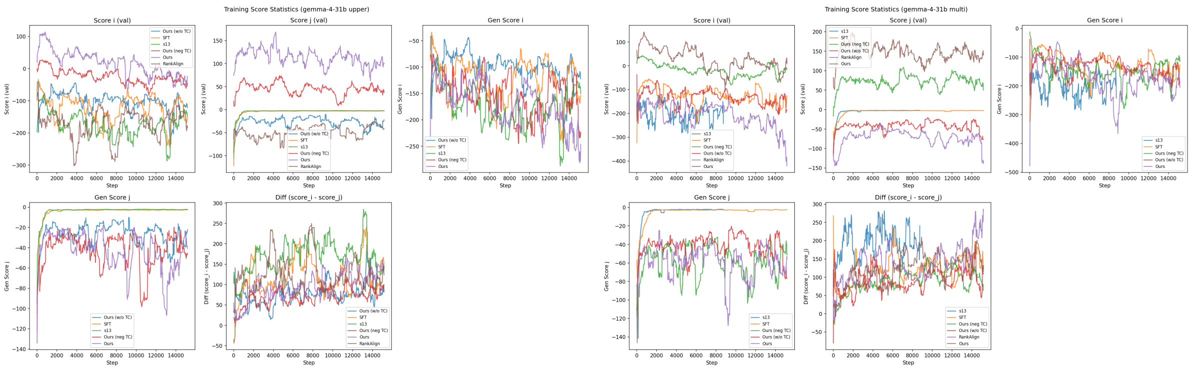

Figure 5: gemma-4-31b training score statistics (gen/val $\mathrm{score}_i/\mathrm{score}_j$, diff): upper (left), multi (right).

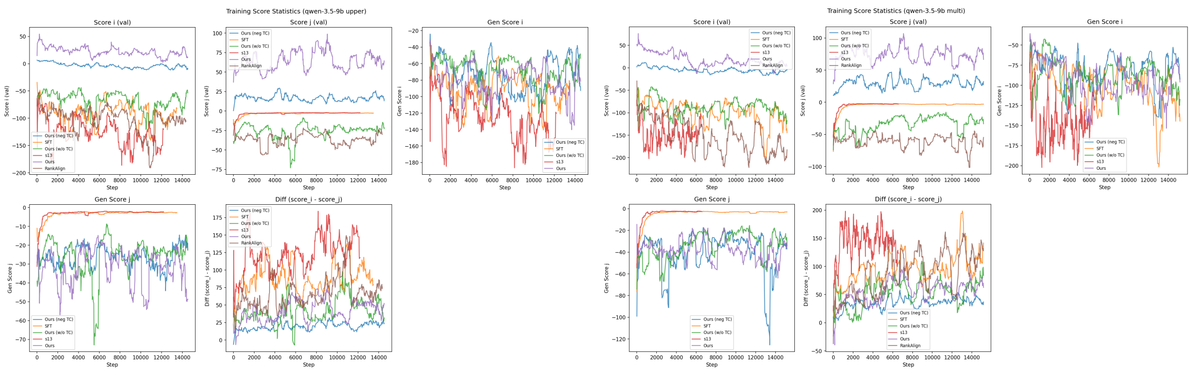

Figure 6: qwen-3.5-9b training score statistics: upper (left), multi (right).

6.4 Held-out test loss (per-checkpoint)

The WandB curves above are training-set losses. To check whether each setting also reduces loss on *held-out* data, we recompute the same three loss components (preference, NLL-G, NLL-V) on the held-out TEST set for each checkpoint (base, epoch 0, 1, 2), exactly as in the original report's held-out validation-loss section. The base row is the untrained model and is drawn as a dashed reference line in each panel.

Held-out set. The 82 test problems are problem-disjoint from the 80 train problems. The base task `humaneval-v2.1correct-{upper,multi}` is train-only, so we aggregate `L_test` across the 82 per-problem test tasks (`--test-task humaneval-{upper,multi}-all`) — the HumanEval analogue of the original's `rosch-all` aggregation — giving 2,219 candidates (1,202 correct, 1,017 incorrect) per dataset; 500 (correct, incorrect) pairs are sampled with a fixed seed for the preference term. Losses use the training-time definitions and the same `make_prompt` (generator / discriminator). Grid: 2 models × 2 datasets × 5 settings (SFT, RankAlign, New+fsx, FLORA-PMI, FLORA-Neg).

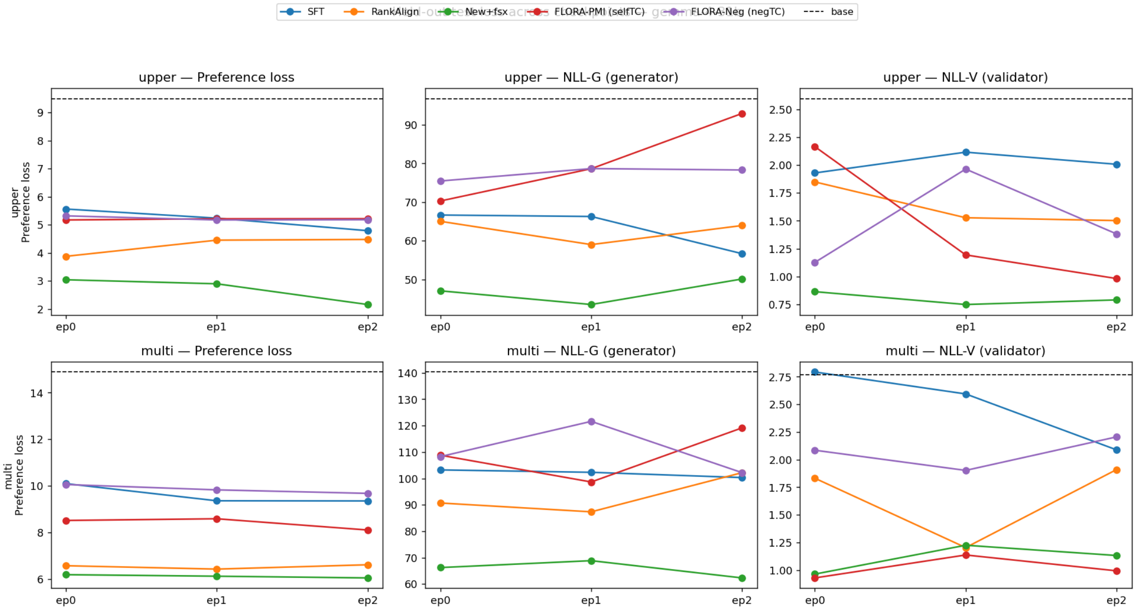

Figure 7: Held-out test loss across checkpoints, gemma-4-31b. Rows: upper / multi. Columns: preference, NLL-G (generator), NLL-V (validator). Dashed black = untrained base; markers at ep0/ep1/ep2.

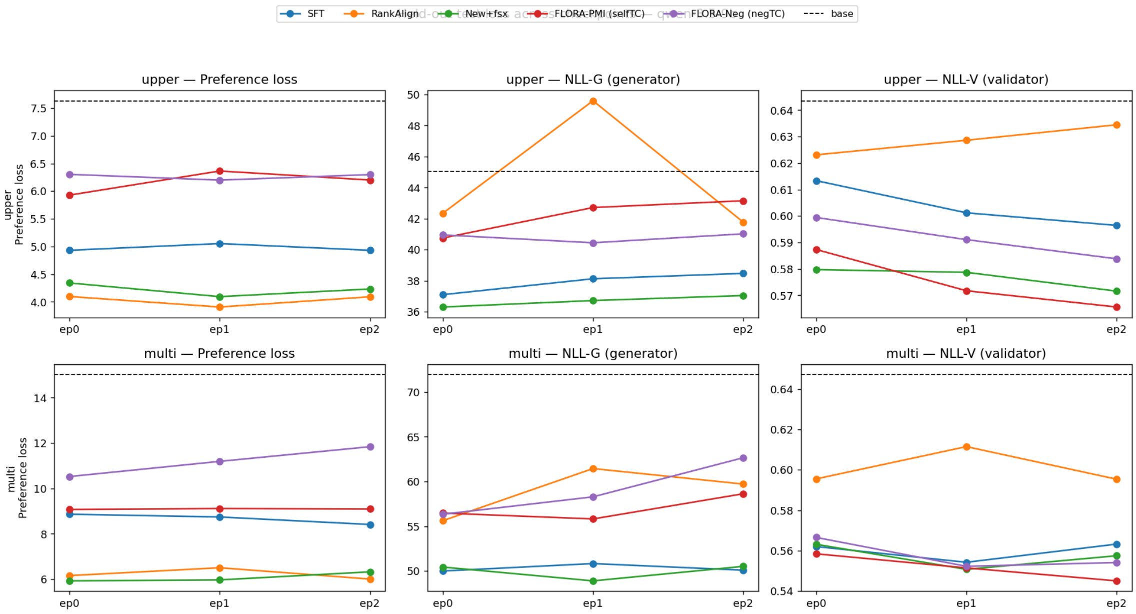

Figure 8: Held-out test loss across checkpoints, qwen-3.5-9b. Same layout.

Reading the figures.

- *No held-out divergence.* Unlike the original IFEval result — where RankAlign's generator NLL exploded to $\sim 1.6 \times 10^4$ on held-out data — here NLL-G stays *controlled* for every setting on both models (gemma-4 / multi: base $\approx 140 \to 62$–120 across settings; qwen comparable). The HumanEval gains are real generalization, not an in-sample optimization artifact.

- *Every method reduces held-out preference loss.* gemma-4 / multi base preference loss is high (14.9) — the multi transforms make correct code atypical by construction — and all five settings cut it on held-out data at epoch 2 (New+fsx 6.0, RankAlign 6.6, FLORA-PMI 8.1, FLORA-Neg 9.7, SFT 9.4). Upper starts lower (base ≈ 7.6 on qwen) and also drops for every setting. No setting regresses *above* base, in contrast to RankAlign on IFEval.

- *NLL-V falls* for most settings as training proceeds (the validator becomes better calibrated on held-out data), with no s2 validator spike of the kind seen on IFEval.

This held-out view reinforces the train-vs-test result (Section 4): the improvements are genuine generalization to unseen problems, and on HumanEval — unlike RankAlign's IFEval instability — no method is unsafe on held-out data.

Table 9: Held-out test loss across checkpoints, gemma-4-31b (all 10 cells × base/ep0/ep1/ep2). Base is the shared untrained reference within each dataset.

Data	Setting	Ckpt	Pref	NLL-G	NLL-V	Total
multi	SFT	base	14.891	140.30	2.771	157.96
		epoch0	10.094	103.23	2.793	116.12
		epoch1	9.357	102.32	2.595	114.28
		epoch2	9.350	100.33	2.089	111.77
multi	RankAlign	base	14.891	140.30	2.771	157.96
		epoch0	6.567	90.70	1.832	99.10
		epoch1	6.421	87.36	1.204	94.99
		epoch2	6.608	102.19	1.910	110.71
multi	New+fsx	base	14.891	140.30	2.771	157.96
		epoch0	6.181	66.26	0.964	73.41
		epoch1	6.116	68.87	1.225	76.21
		epoch2	6.042	62.34	1.133	69.51
multi	FLORA-PMI (selfTC)	base	14.891	140.30	2.771	157.96
		epoch0	8.512	108.80	0.930	118.24
		epoch1	8.585	98.66	1.137	108.38
		epoch2	8.102	119.15	0.994	128.24
multi	FLORA-Neg (negTC)	base	14.891	140.30	2.771	157.96
		epoch0	10.056	108.27	2.085	120.41
		epoch1	9.826	121.61	1.903	133.34
		epoch2	9.675	102.20	2.206	114.08
upper	SFT	base	9.496	96.81	2.595	108.91
		epoch0	5.559	66.67	1.929	74.16
		epoch1	5.234	66.29	2.115	73.64
		epoch2	4.788	56.66	2.007	63.46
upper	RankAlign	base	9.496	96.81	2.595	108.91
		epoch0	3.874	65.04	1.848	70.76
		epoch1	4.450	59.01	1.528	64.99
		epoch2	4.476	63.95	1.502	69.93
upper	New+fsx	base	9.496	96.81	2.595	108.91
		epoch0	3.039	47.04	0.865	50.95
		epoch1	2.895	43.50	0.750	47.15
		epoch2	2.156	50.11	0.792	53.06
upper	FLORA-PMI (selfTC)	base	9.496	96.81	2.595	108.91
		epoch0	5.171	70.30	2.164	77.64
		epoch1	5.213	78.66	1.195	85.07
		epoch2	5.216	92.97	0.983	99.17
upper	FLORA-Neg (negTC)	base	9.496	96.81	2.595	108.91
		epoch0	5.317	75.48	1.128	81.92
		epoch1	5.170	78.69	1.963	85.83
		epoch2	5.179	78.33	1.383	84.89

Table 10: Held-out test loss across checkpoints, qwen-3.5-9b (all 10 cells × base/ep0/ep1/ep2).

Data	Setting	Ckpt	Pref	NLL-G	NLL-V	Total
multi	SFT	base	15.019	71.95	0.647	87.62
		epoch0	8.864	49.99	0.562	59.42
		epoch1	8.742	50.82	0.554	60.12
		epoch2	8.411	50.09	0.563	59.06
multi	RankAlign	base	15.019	71.95	0.647	87.62
		epoch0	6.157	55.61	0.595	62.37
		epoch1	6.504	61.45	0.611	68.56
		epoch2	6.005	59.71	0.595	66.31
multi	New+fsx	base	15.019	71.95	0.647	87.62
		epoch0	5.925	50.43	0.563	56.92
		epoch1	5.968	48.89	0.551	55.41
		epoch2	6.323	50.51	0.557	57.40
multi	FLORA-PMI (selfTC)	base	15.019	71.95	0.647	87.62
		epoch0	9.075	56.47	0.558	66.10
		epoch1	9.113	55.81	0.551	65.48
		epoch2	9.094	58.63	0.545	68.27
multi	FLORA-Neg (negTC)	base	15.019	71.95	0.647	87.62
		epoch0	10.524	56.34	0.567	67.43
		epoch1	11.193	58.28	0.552	70.03
		epoch2	11.840	62.66	0.554	75.05
upper	SFT	base	7.634	45.04	0.644	53.32
		epoch0	4.932	37.10	0.613	42.65
		epoch1	5.054	38.13	0.601	43.78
		epoch2	4.932	38.47	0.596	44.00
upper	RankAlign	base	7.634	45.04	0.644	53.32
		epoch0	4.097	42.35	0.623	47.07
		epoch1	3.907	49.60	0.629	54.13
		epoch2	4.091	41.77	0.634	46.50
upper	New+fsx	base	7.634	45.04	0.644	53.32
		epoch0	4.342	36.31	0.580	41.23
		epoch1	4.095	36.72	0.579	41.40
		epoch2	4.233	37.05	0.572	41.85
upper	FLORA-PMI (selfTC)	base	7.634	45.04	0.644	53.32
		epoch0	5.928	40.75	0.587	47.27
		epoch1	6.364	42.72	0.572	49.65
		epoch2	6.201	43.16	0.566	49.92
upper	FLORA-Neg (negTC)	base	7.634	45.04	0.644	53.32
		epoch0	6.304	40.95	0.599	47.86
		epoch1	6.200	40.45	0.591	47.24
		epoch2	6.300	41.02	0.584	47.91

Reproducibility (held-out loss). Compute: `scripts/compute_val_loss_he.py` (a copy of `compute_val_loss.py`, original untouched; explicit `--epN-path`, adapter-aware loading [gemma-4 LoRA on base, qwen merged], held-out aggregation via `--test-task humaneval-{upper,multi}-all`, resumable per-checkpoint CSV). Launcher: `scripts/mll_he_val_loss.sbatch` + `launch_he_val_loss.sh` (20 Slurm jobs 45247–45266; gemma-4 gpu:4, qwen gpu:1; one per-job CSV each). Build: `scripts/build_he_valloss_section.py`. `--max-pairs 500`, `--seed 42`.

7 Diagnostics: Dynamics / Scatter / Delta / Per-problem

7.1 Per-epoch training dynamics (TRAIN, per-problem)

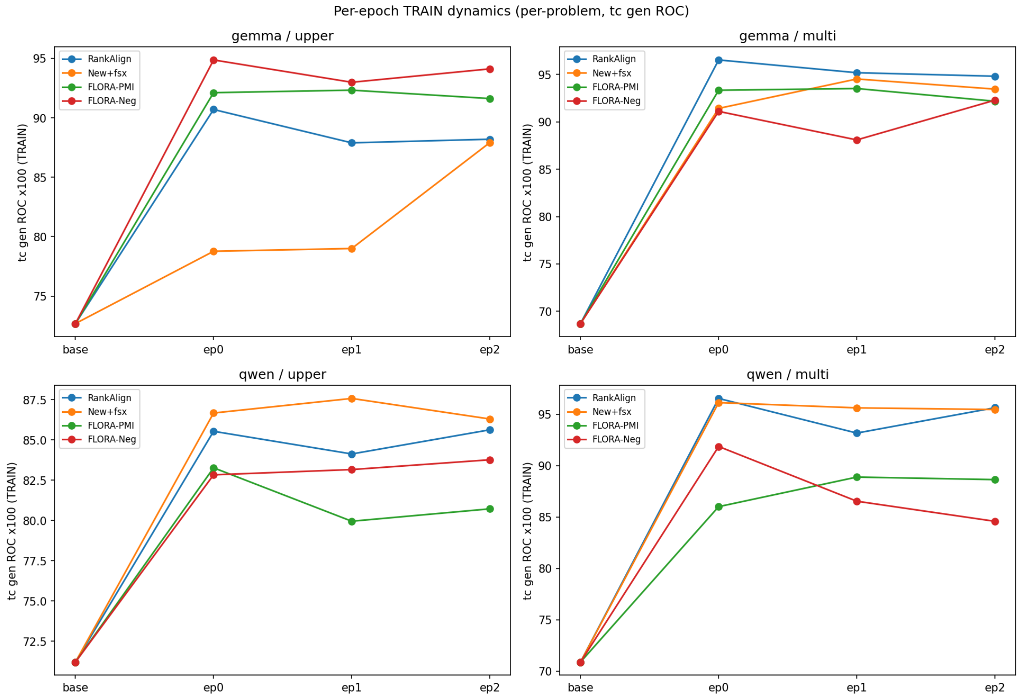

Figure 9: Per-problem tc gen ROC vs. checkpoint (base → ep0/1/2) on the TRAIN problems, each method in its natural eval mode. TC methods (s4/s7) climb fastest and highest.

7.2 Generator–validator scatter (TRAIN, epoch 2)

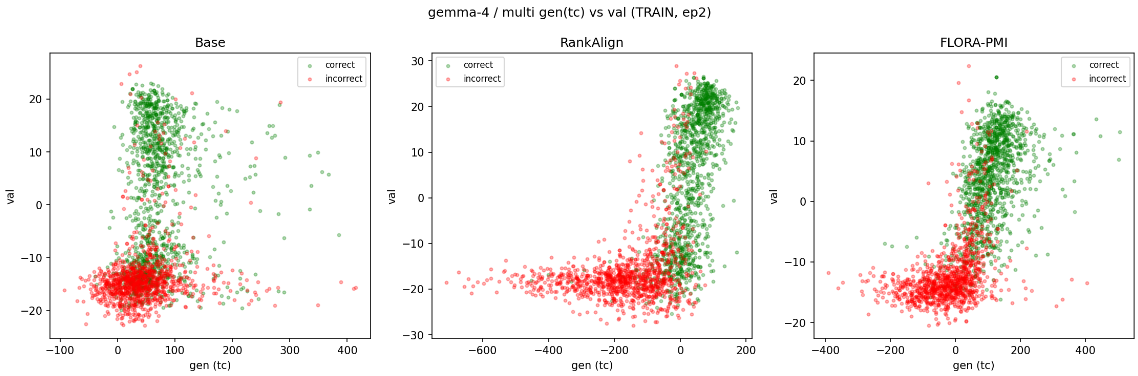

Figure 10: gen (tc) vs. validator score, correct (green) / incorrect (red), gemma-4 / multi, Base vs RankAlign vs FLORA-PMI. Training separates the classes in both gen and val space.

7.3 Score-delta distributions (TRAIN, epoch 2)

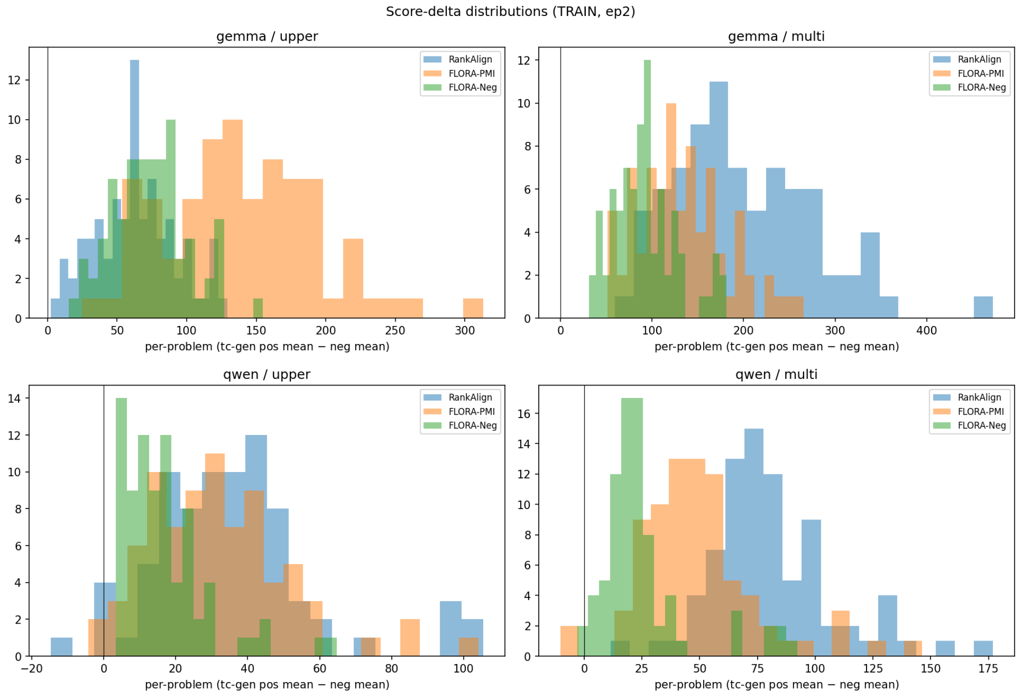

Figure 11: Per-problem (mean tc-gen of correct − mean of incorrect). Mass above 0 means the generator separates correct from incorrect; wider/cleaner separation for the TC methods.

7.4 Per-problem effect distribution (TEST)

HumanEval has no category structure, so in place of the original per-category analysis we show the *per-problem* distribution of the neg-TC effect (s7 − s3).

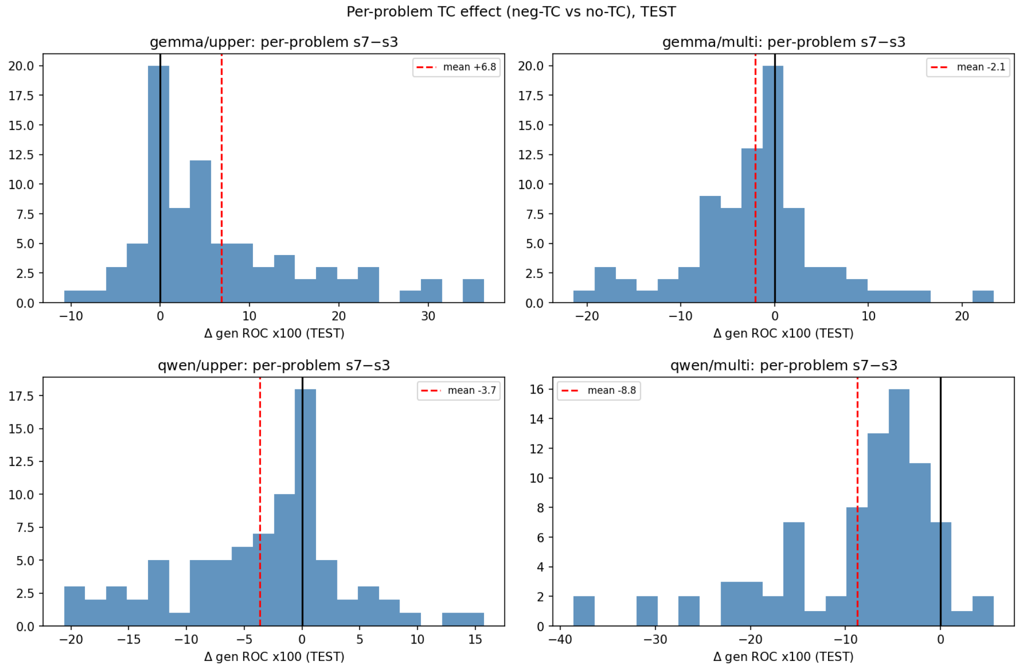

Figure 12: Per-problem Δ tc gen ROC (FLORA-Neg $-$ New+fsx) on the TEST problems. The sign/spread shows TC's effect is heterogeneous across problems and regime-dependent.

8 Conclusions

1. **The upper-vs-multi flip is real and gemma-4-specific**: our TC methods (s7, s4) beat RankAlign on gemma-4 *upper* but lose on *multi*; on qwen RankAlign wins both.
2. **It is NOT overfitting.** With the correct per-problem train-vs-test (80 train vs 82 test problems, scored identically), the train→test gap is within ± 2 for all gemma-4 methods and *negative* for qwen (test $\geq$ train). The earlier $\Delta = +8.9$ "overfitting" was an artifact of comparing a global-pool train metric to a per-problem test metric; recomputed per-problem it is $+2.0$ (Section 3).
3. **The test-set differences are an eval-time effect**: on multi the raw generator score is near chance (the ΔlogP-selected stylizations make correct code atypical) and the typicality correction does the heavy lifting (Section 5); the RankAlign-vs-ours ordering follows from how each method's training interacts with that correction, not from overfitting.

Scope note: this report mirrors all sections of the original `analysis/report.tex` (Q1 comparison + unified bar; WandB loss curves, components, and score statistics; concordance; Spearman/Pearson; score-delta; training dynamics; gen-vs-val scatter; TC comparison; train-vs-test; held-out test loss). One deviation: the per-category analysis is replaced by a per-problem effect distribution (HumanEval has no categories).

Reproducibility. Tables built by `analysis/scripts/build_he_report_tables.py` from `docs/he_harvest_` (test) and `docs/he_trainset_2026-06-19/humaneval_TRAIN_metrics_aggregated.csv` (train, gemma s2/s4). trs2 train-set batch: `scripts/mll_he_trainset_eval_v2.sbatch` + `launch_he_trainset_extra` model origins in `docs/he_trainset_eval_model_origins.md`.